HappyRobot End-of-Turn Detection

1. Executive summary

This submission implements and evaluates a small audio-only end-of-turn (EoT) baseline, exposes it through FastAPI, and includes a Docker deployment. It uses the public `pipecat-ai/smart-turn-data-v3-train` dataset, 45 deterministic acoustic features, and a Logistic Regression classifier. The selected threshold is `0.3691340835714273`, chosen on validation data.

On the 750-example held-out test split, F1 is 0.6538 and precision is 0.5121. The model has 323 false positives, a serious interaction risk because an incorrect endpoint can interrupt a user. The saved Docker verification shows the image, startup, readiness, real inference, invalid-audio response, and restart checks passing. These results establish a working take-home baseline, not production-quality accuracy or production deployment readiness.

2. Problem definition

The assessment defines EoT as deciding when a user has finished speaking and expects the agent to respond. It asks for English support (multilingual support is a bonus), low latency (ideally below 100 ms per request), a trained, evaluated, and served model, a stress test with local throughput/latency findings, a Dockerfile, a production monitoring/improvement plan, a short solution document, documented code, and a video no longer than 10 minutes.

The target is the dataset's boolean `endpoint_bool`: `True` means the current audio is labeled as an end-of-turn. This is a turn-taking decision, not just speech-versus-silence detection.

3. Dataset and preparation

- Source: `pipecat-ai/smart-turn-data-v3-train`. The inspection and preparation scripts now pin new runs to revision `909e604f990651631a596f5cb03358b3ee8fbad0`; the existing prepared split files do not record their historical source revision.
- Preparation uses Hugging Face streaming and bounded scans, selecting 5,000 valid examples from a scan bounded at 20,000 rows; it does not download the full corpus.
- The selected set has 2,500 positive and 2,500 negative labels. Audio is checked by reopening and decoding it; the dataset summary reports durations from 0.43 to 25.56 seconds, mean 7.25 seconds, and 16 kHz audio.
- `synthetic=True` occurs on 4,892 of the 5,000 selected examples (97.84%). There are 23 represented language codes, but counts are uneven; English has 539 selected examples.
- `data/processed/selected_samples.csv` matches the metadata copy. All split CSV references resolve to audio files. The split CSVs reference 5,000 unique audio files; the audio directory currently contains 9,998 files, leaving 4,998 unreferenced files. They may be legacy files; their provenance was not established, and they were not removed.

4. Data splitting strategy

`prepare_dataset.py` performs a deterministic, label-stratified 70/15/15 split with seed 42:

Split	Samples	Positive	Negative
Train	3,500	1,750	1,750
Validation	750	375	375
Test	750	375	375

An independent audit of the persisted CSVs found unique IDs within each split, zero ID overlap between splits, and all referenced audio present. The split is sample-level, not speaker-level: the inspected dataset schema has no speaker, recording, or conversation grouping ID. No cross-validation or grouped/temporal evaluation is implemented.

5. Feature engineering

`scripts/extract_features.py` is shared by training, offline prediction, and HTTP serving. It decodes audio, mixes channels to mono, resamples to 16 kHz, and computes 45 ordered features: duration; RMS and short-time-energy summaries; low-energy fraction; zero-crossing rate; spectral centroid, bandwidth, rolloff, and flux summaries; and mean/standard deviation for 13 MFCCs. Short-time framing uses 25 ms windows, a 10 ms hop, and a 512-point FFT. Invalid or non-finite feature output is rejected.

Language, labels, IDs, filler flags, synthetic markers, and source metadata are not model inputs. Metadata is reserved for validation error analysis.

6. Baseline selection

`scripts/train_baseline.py` compares a most-frequent DummyClassifier, Logistic Regression, and a 300-tree Random Forest. The learned pipelines use median imputation and standard scaling; the seed is 42. The threshold is chosen separately for each learned candidate using validation F1, with ties resolved by precision and then the higher threshold. Logistic Regression is selected because its validation F1 (0.6705) exceeds Random Forest (0.6673).

7. Training and validation results

Validation results from the saved reports:

Model	Threshold	Accuracy	Precision	Recall	F1	FP	FN
Dummy, most frequent	n/a	0.5000	0.0000	0.0000	0.0000	0	375
Logistic Regression	0.369134	0.5427	0.5240	0.9307	0.6705	317	26
Random Forest	0.245444	0.5013	0.5007	1.0000	0.6673	374	0

The selected threshold is stored in `models/baseline/model_config.json`. Training code opens only train and validation CSVs; it does not load the test split. The test evaluator refuses to proceed unless the model config is marked finalized, then loads the saved threshold without selecting or tuning it.

The split and source code provide evidence of held-out test use after finalization, but the repository does not contain an immutable experiment ledger proving the historical order of every run. Results should be described as code-enforced validation/test separation, not independently audited run provenance.

8. Held-out test results

The saved test report, persisted predictions, and a read-only in-memory inference pass using the saved model agree. The saved model reproduced all 750 prediction labels; the maximum probability difference from the persisted CSV was `8.33e-17`. No artifact or report was written during this check.

Samples	Accuracy	Precision	Recall	F1	FP	FN	ROC-AUC	Average precision
750	0.5213	0.5121	0.9040	0.6538	323	36	0.5750	0.5751

Confusion matrix, actual rows `[negative, positive]`, predicted columns `[negative, positive]`: `[[52, 323], [36, 339]]`.

The selected operating point favors recall but has poor precision. The assessment asks for English support; a diagnostic slice of the saved test predictions contains 82 English examples (42 positive, 40 negative), with precision 0.507, recall 0.833, and F1 0.631 (34 FP, 7 FN). This is a small subgroup and is not a robust English-only qualification or a separately tuned model. No confidence intervals are reported.

An additional, disjoint English-only evaluation used an approved 800-example cohort (400 positive, 400 negative) from the same dataset source at pinned revision `909e604f990651631a596f5cb03358b3ee8fbad0`. The existing model and threshold were frozen; no retraining or tuning was performed. It yielded accuracy 0.5075, precision 0.5045, recall 0.8400, F1 0.6304, and average precision 0.5654; its confusion matrix is `[[70, 330], [64, 336]]`. Stratified bootstrap 95% intervals are in `reports/english_evaluation/run_seed42_909e604f/metrics.json`. This cohort is balanced and drawn from the same source, so it is not an external-domain or production-prevalence estimate and does not establish robust English generalization.

9. Error analysis

A false positive predicts the user is done when they are still speaking and can cut off their turn. A false negative misses a true endpoint and can add silence or response delay. The 323 test false positives make interruption risk the primary quality concern.

Validation error analysis is explicitly descriptive. For example, 199 of 233 validation examples marked `endfiller=True` were errors, all false positives. This association does not establish that the filler caused an error. Every split is 16 kHz, so the data cannot compare sample-rate effects. The validation analysis does not prove that pauses caused individual mistakes.

10. Model serving architecture

`scripts/serve.py` loads the existing model/config once in FastAPI lifespan and checks the feature schema and finalized threshold. It calls the same feature extractor and prediction function as offline inference. The service does not download data or train a model.

Endpoints:

- `GET /health`: process health.
- `GET /ready`: model readiness; returns 503 if the model bundle is absent.
- `POST /predict`: multipart field `audio`, accepting WAV or FLAC.

Uploads are limited to 20 MiB, 0.1-30 seconds, 8-96 kHz, and 1-8 channels. Extension, media type, container, readability, and decoded features are validated. The seekable FastAPI upload spool is reused; a temporary file is created only for a non-seekable stream and removed after the request. The API returns the boolean prediction, probability, threshold, model/version, audio metadata, and server-side timing spans. `upload_ms` starts after FastAPI has parsed the multipart body and measures upload inspection/opening, not client transfer or multipart spooling. `server_processing_ms` is not the complete client round trip.

11. API usage

From `happyrobot-turn-detection/`:

```
python -m pip install -r requirements-serving.txt
python -m uvicorn serve:app --app-dir scripts --host 127.0.0.1 --port 8000
```

In another terminal:

```
python scripts\serve_client.py --audio path\to\sample.wav
```

The client sends a single WAV/FLAC request and prints the response and client-observed latency.

12. Performance and latency

The offline latency report measures 750 validation examples with a warm file cache. It reports model-only prediction at 1.21 ms mean and 2.18 ms P95; audio loading/decoding, features, and prediction at 17.17 ms mean and 28.45 ms P95. Model startup is excluded.

The original HTTP stress test reuses one synthetic 0.5-second, 16 kHz WAV file (16,044 bytes) over a warm connection pool, sending 100 requests per concurrency level. All 400 requests succeeded. Its original report remains unchanged at `reports/serving/performance.json`.

Concurrency	Throughput	HTTP P95	Model-only P95
1	51.45 req/s	23.82 ms	3.08 ms
2	60.92 req/s	39.45 ms	2.61 ms
4	76.55 req/s	66.60 ms	3.39 ms
8	95.28 req/s	111.52 ms	3.11 ms

HTTP latency is client-measured from request dispatch through response body receipt; model-only timing excludes audio I/O and feature extraction. These are local short-clip results, not cold-start or production measurements. The concurrency-8 HTTP P95 exceeds the assessment's ideal 100 ms target; the target is not met in every tested condition. The 0.5-second benchmark clip is shorter than the dataset's reported 7.25-second mean.

The subsequent serving optimization preserved the 45 feature values, probabilities, and predictions exactly on the same 0.5-second WAV, 6.75-second WAV, and 6.75-second FLAC inputs. The following separate run used eight Uvicorn workers and 100 requests per concurrency level; it does not replace the original benchmark:

Audio	Concurrency 8 throughput	Mean	Median	P95	P99
0.5-second WAV	122.46 req/s	62.04 ms	57.57 ms	89.19 ms	122.23 ms
6.75-second FLAC	69.60 req/s	111.38 ms	107.40 ms	146.41 ms	156.66 ms

All 100 requests succeeded for both inputs. This later run is below 110 ms P95 for the short clip, but not for the longer representative clip; the assessment's ideal latency target is therefore not met for typical-duration audio. Reports are kept separately in `reports/serving/phase2_followup_20261107/`. The existing reports do not record worker count in their JSON; future benchmark runs accept `--server-workers` and record it.

13. Docker deployment and verification

`Dockerfile` installs serving dependencies and `libsndfile`, copies only serving modules plus the finalized model/config, runs as a non-root user, and defines a `/ready` health check. `.dockerignore` excludes data, reports, and tests from the build context.

The recorded Docker verification (2026-11-07) confirmed:

- Image `happyrobot-eot:latest` built successfully (467,206,428 bytes).
- Container started with port 8000 mapped; `/ready` returned 200 and Docker health became `healthy`.
- A prepared 6.16-second FLAC produced a `logistic_regression` prediction (`endpoint_probability=0.3934`, threshold `0.369134`).
- Malformed WAV audio returned HTTP 422.
- Stop/start returned the container to ready/healthy; it was left stopped.
- No dataset download or retraining occurred.

The full recorded verification and PowerShell reproduction steps are in `serving_and_performance.md`. Docker was not rebuilt or restarted during this final audit.

14. Monitoring strategy

`monitoring_and_improvement.md` proposes service/error/latency metrics, resource and saturation signals, delayed-label quality evaluation, drift checks, privacy controls, alert thresholds, human review, and shadow/canary rollout. It explicitly distinguishes these plans from current implementation. The API exposes health/readiness and request timing fields, but there is no Prometheus exporter, Grafana dashboard, Alertmanager integration, delayed-label store, or live model-quality monitor in this codebase.

15. Limitations and risks

- The model has low test precision and 323 false positives; it is not ready to make autonomous endpoint decisions in production without product safeguards and better evidence.
- The selected set is small and 97.84% marked synthetic; a balanced test set does not establish performance at production prevalence.
- The English test subgroup is only 82 examples. Multilingual coverage is uneven and not a reliable per-language guarantee.
- There are no speaker/conversation IDs, so the random splits cannot prove speaker-independent generalization. There is no cross-validation or temporal/grouped test set.
- The benchmark is one synthetic short clip on one development machine; concurrency 8 exceeds 100 ms P95.
- There is no pinned data/training dependency manifest. The only requirements file is for serving and does not include Hugging Face `datasets`.
- The audio directory contains 4,998 files not referenced by current split CSVs. They were not changed or removed; their provenance should be resolved before deciding what belongs in a submission package.
- The project folder is currently untracked by the parent Git repository. No files were staged or committed during this audit.

16. Future improvements and assessment discussion

The current audio-only baseline was a practical first step: audio is directly available to an endpointing system, fixed acoustic features are inexpensive, and a compact classifier can be served without waiting for transcription. It is a baseline choice, not evidence that audio-only is universally better.

VAD answers whether speech is present or where speech segments occur. Endpointing decides whether the user intends to yield the conversational turn. A pause can occur inside an utterance, and fillers or continued speech after a pause can make VAD-only stopping interrupt the user. VAD can supply useful speech/pause boundaries to an endpoint model but is not by itself a complete turn-taking decision.

Potential additions include temporal acoustic features, incremental ASR text, prosody, punctuation/linguistic completion, speaker and agent-turn context, and multimodal signals. An ASR model such as Whisper or Parketeer could provide partial transcripts and text-derived completion cues, or feed a text/audio classifier; this is proposed work, not implemented here. ASR adds latency, compute, transcription errors, and privacy considerations, so it should be evaluated against the current audio baseline and the 100 ms target.

In a voice-agent architecture, a VAD/audio stream can feed an endpoint scorer; a turn manager can combine its score with partial transcript, conversation state, and interruption policy before dispatching to the LLM. The system should preserve barge-in/continuation behavior and avoid treating one threshold decision as definitive when confidence is low.

Production improvement should start with privacy-approved, representative audio and delayed, reviewed labels. Keep random samples as well as targeted uncertain/interrupt-prone cases, track label delay and cohort mix, evaluate false positives/negatives on temporal and speaker-grouped sets when possible, and compare candidates offline before shadow/canary rollout. Do not automatically retrain or retune from drift alerts alone.

17. Reproduction instructions

The installed audit environment had Python 3.10.11 and `datasets==5.1.0`. `requirements-serving.txt` pins serving and model dependencies; it does not pin the preparation package. On a clean copy, install:

```
python -m pip install -r requirements-training.txt
```

Dataset inspection and preparation are bounded streaming operations:

```
python scripts\inspect_dataset.py --max-inspection 1000
python scripts\prepare_dataset.py --target-samples 5000 --seed 42 --max-inspection 20000 --max-scan 20000
```

Preparation writes selected audio and CSVs under `data/`, and rewrites `docs/dataset_summary.md`; run it only in a clean copy if the current prepared dataset must be preserved. Training and evaluation also write model/report artifacts, so reproduce them only in a clean copy:

```
python scripts\train_baseline.py --seed 42
python scripts\evaluate_baseline.py
```

The existing tests can be run without rebuilding the saved baseline:

```
python -m unittest discover -s tests -v
```

For serving and Docker commands, see `serving_and_performance.md`. Do not rerun training or overwrite saved evaluation reports merely to reproduce the already recorded results.